

Introduction to Arduino

September 3, 2024

Chapter 1

IDE Installation

To become familiar with the use of Arduino boards, you will perform several experiments using this board. In order to do this, you have to procure an Arduino Nano board and a breadboard for making connections to it. To develop applications using these boards, you have to install an Integrated Development Environment (IDE) for Arduino on your laptop. Procedure for this is described in this document. After installation of IDE, you can proceed to perform the simple experiments described here to become familiar with the use of Arduino boards.

Arduino programs are called “sketches”. These sketches are compiled on the host PC and the resulting binary program is uploaded to the Arduino board by the IDE using the USB interface.

Material in this document has been compiled from various tutorials available on the net for Arduino.

1.1 Arduino IDE Installation

Installation of the Integrated Development Environment (IDE) for Arduino requires

- i) downloading the software and
- ii) installing it using the installation utilities of your operating system.

1.2 Downloading Arduino IDE

IDE software can be downloaded from the Arduino Software page:

<https://www.arduino.cc/en/software>

Operating System version requirements for running the IDE are:

Windows: Win 10 and newer, 64 bits, or

Mac OS X: Version 10.14: “Mojave” or newer, 64 bits, or

Linux: 64 bit distributions

Arduino IDE 2.x is an open-source project. It is a big step from its sturdy predecessor, Arduino IDE 1.x, and comes with revamped User Interface or UI, improved board and library manager, debugger, auto-complete feature and much more. Currently, the latest version of the IDE is 2.3.2 (This may change with time).

To download the Editor, you need to visit the Arduino software page. Go to:

<https://www.arduino.cc/en/software>

and click on the Download button appropriate for your operating system.

1.3 Installing the IDE

Installation procedure is operating system dependent.

Windows: To install Arduino IDE 2.x on a Windows computer, simply run the file downloaded from the software page. Follow the instructions in the installation guide. The installation may take several minutes.

Some windows installations require that you install a special serial port driver (CH 340) before you can communicate with Arduino cards through the USB interface. This driver can be installed from:

https://www.wch-ic.com/downloads/CH341SER_ZIP.html

A tutorial for installing it is available at:

<https://learn.sparkfun.com/tutorials/how-to-install-ch340-drivers/all>

MAC OS: To install the Arduino IDE 2.x on a MAC OS computer, simply copy the downloaded file into your application folder.

Linux: To install Arduino IDE 2.3.2 on Linux, first download the file:

`arduino-ide_2.3.2_Linux_64bit.zip` from the software page and place it in a directory of your choice. The file is compressed and you have to extract it in a suitable folder, remembering that it will be executed from there. For this, open a terminal window in this directory and unzip the file:

`sudo unzip arduino-ide_2.3.2_Linux_64bit.zip`. This will extract all the necessary files from the zip archive. You will find a directory called `arduino-ide_2.3.2_Linux_64bit`.

Change to this directory by:

`cd arduino-ide_2.3.2_Linux_64bit`.

Running Arduino-IDE from this directory (for example, by double clicking on the file name) will start the IDE program.

Optional step: add a link to the executable file

If you have a bin directory added to your path, you can place a link to this executable file in your bin directory.

`cd <path to your bin directory>`

`ln -s <arduino directory path>/arduino-ide_2.3.2_Linux_64bit/arduino-ide ide`

Now you will be able to launch the IDE from any directory. This step is optional.

Arduino programs (called “sketches”) are compiled on the host PC and uploaded using a serial connection to the Arduino board. Since Linux is a multi-user OS, access to common resources like the serial or USB port is controlled by the OS – otherwise multiple users may use a resource simultaneously and destroy each others’ data. Hence, access to common resources has to be obtained from the system administrator before you can use these.

When you start the IDE on your PC, you have to select a board (so that the “sketch” will be compiled according to specifics of your board) and a port, which will be used to communicate with your board.

It is possible that when you upload a sketch - (after selecting your board and the serial port), - you get an error:

“Error opening serial port ...”, or

“can’t open device ”/dev/ttyUSB0”: Permission denied

If you get this error, you need to set serial port permissions. To do this:

Open a terminal and run `ls -l` on the problem device. For this, type:

```
ls -l /dev/ttyACM*
```

or

```
ls -l /dev/ttyUSB*
```

you will get something like:

```
crw-rw— 1 root dialout 188, 0 5 apr 23.01 ttyACM0
```

or

```
crw-rw— . 1 root dialout 188, 0 Nov 8 11:44 /dev/ttyUSB0
```

The “0” at the end of ACM/USB might be a different number depending on the actual serial line of USB port being used, or multiple entries might be returned. We want to find the group owner of the tty – in this case, it is “dialout”.

Now we just need to add our user to the group: This is done by:

```
sudo usermod -a -G dialout <username>
```

where <username> is your Linux user name. This will add you to the group “dialout” which owns this serial line. You will need to log out and log in again for this change to take effect.

After this procedure, you should be able to proceed normally and upload the sketch to your board or use the Serial Monitor.

1.4 Compiling and uploading Arduino programs

Arduino programs are called “sketches”. These sketches are compiled on a PC or laptop and then the binary is uploaded to the Arduino card using the USB port.

If you are able to compile Arduino code but the IDE cannot upload the code to the Arduino card, check if the proper serial driver is installed if you are using Windows and proper permissions are set on the serial line if you are using Linux. If you are still unable to upload the code, it is possible that the processor used on your card has an old version of bootloader code. (Boot loader is a pre-loaded program on the micro-controller which enables it to use the USB port to receive compiled programs and to store them in the program memory). In this case, you have to set the processor type as “ATmega328P (Old Bootloader)”.

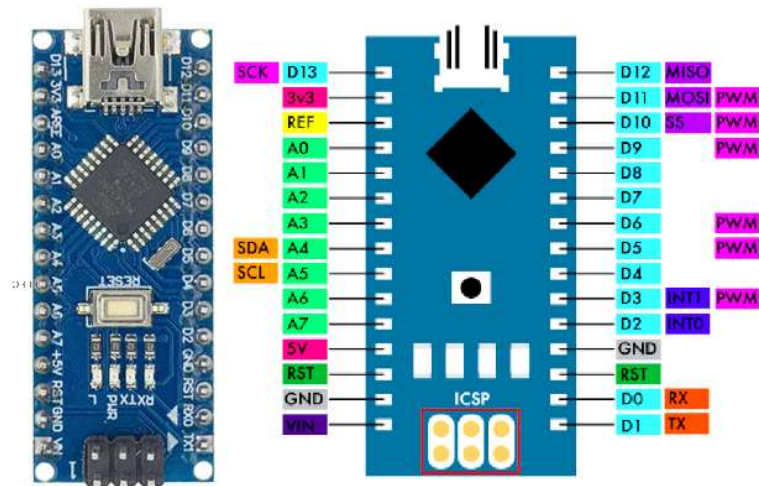
You can do this by going to the Tools tab on the IDE and clicking on “processor”. Choose “ATmega328P (Old Bootloader)” from the choices offered. Now the upload will go through.

Chapter 2

Arduino Hardware and Library Functions

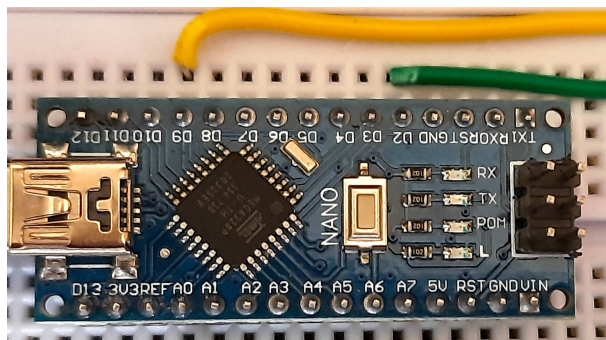
2.1 Arduino board pinout

2.1.1 Arduino Nano board



Picture on the left shows an Arduino Nano board.

Pins are brought out to pins on the sides of the board. Pin names are silk-screened on the board next to the pins.



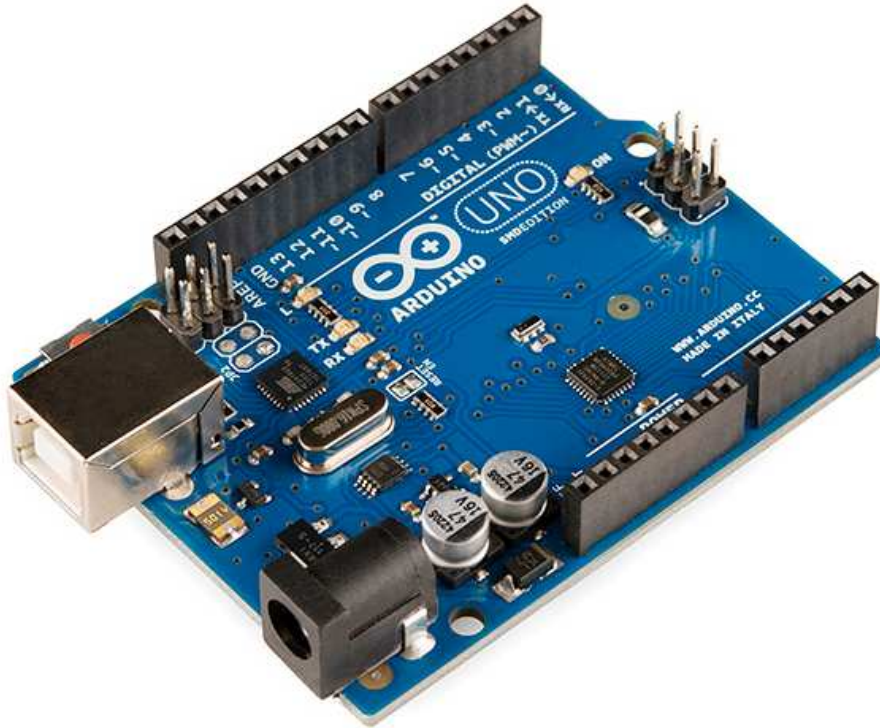
The nano board can be plugged into a breadboard just like a large IC.

The board can be powered by the USB interface. (It uses a mini-USB connector). It can also be powered by supplying an external DC voltage between 7 and 12 V, fed to its V_{in} pin.

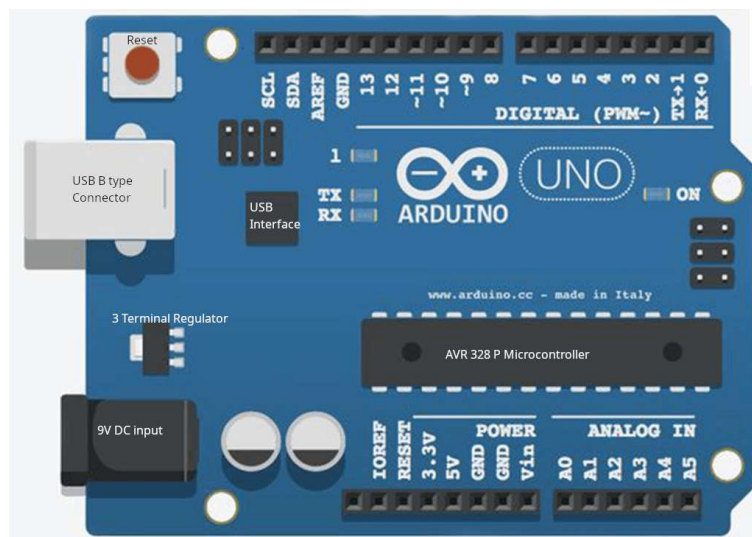
2.1.2 Arduino Uno board

The picture below shows an Arduino Uno board.

Pins are brought out to the connectors on the sides of the board and pin names are silk-screened on the board next to the connector sockets.



The Uno board has an additional connector, which allows it to be powered by a wall plug type DC source supplying a DC voltage in the range 7 to 12 V.



Pin names for Uno and Nano are practically identical.

Pinout for Arduino Nano		
Pin Category	Pin Name	Details
Power:	Vin	Input voltage to Arduino when using an external power source.
	5 V	Regulated power supply used to power microcontroller, other components
	3.3 V	3.3 V supply generated by on-board voltage regulator. Max current: 50 mA.
	GND	ground pins.
I/O Pins:		
Digital Pins	0 - 13	Digital input or output pins
Pulse Width Modulation	3, 5, 6, 9, 11	8-bit PWM output.
Analog Pins	A0 – A7	Used to provide analog input in the range of 0-5 V
Analog Ref	AREF	To provide reference voltage for input voltage.
Serial	0(Rx), 1(Tx)	Receive and Transmit TTL serial data.
SPI	11 (MOSI), 12 (MISO) 10 (SS), 13 (SCLK)	Used for SPI communication
I2C interface	A4 (SDA), A5 (SCA)	for “two wire interface” (TWI) or I2C communication
External Interrupts	2, 3	To trigger an interrupt
Reset	Reset	Resets the microcontroller.
Inbuilt LED	13	To turn on the inbuilt LED

2.2 Language Reference

Arduino programming language can be divided in three main parts: functions, values (variables and constants), and structure.

2.3 functions

2.3.1 For controlling the Arduino board and performing computations:

Digital I/O: `digitalRead( )`, `digitalWrite( )`, `pinMode( )`

Analog I/O: `analogRead( )`, `analogReference( )` `analogWrite( )`

on Zero, Due and MKR Family: `analogReadResolution( )`, `analogWriteResolution( )`

2.3.2 Advanced I/O:

`noTone( )`, `tone( )`

`pulseIn( )`, `pulseInLong( )`, `shiftIn( )`, `shiftOut( )`

`Time`, `delay( )`, `delayMicroseconds( )`, `micros( )`, `millis( )`,

2.3.3 Math:

abs(), constrain(), map(), max(), min(), pow(), sq(), sqrt()

Trigonometry:

cos(), sin(), tan()

2.3.4 Characters:

isAlpha(), isAlphaNumeric(), isAscii(), isControl(), isDigit(), isGraph()

isHexadecimalDigit(), isLowerCase(), isPrintable(), isPunct(), isSpace()

isUpperCase(), isWhitespace()

2.3.5 Random Numbers:

random(), randomSeed()

2.3.6 Bits and Bytes:

bit(), bitClear(), bitRead(), bitSet(), bitWrite()

highByte(), lowByte()

2.3.7 Interrupt controls:

interrupts(), noInterrupts()

2.3.8 External Interrupts:

attachInterrupt(), detachInterrupt()

2.3.9 Communication:

Serial, SPI, Stream, Wire, USB, Keyboard, Mouse

2.4 Variables:

Arduino data types and constants:

Constants:

HIGH | LOW

INPUT | OUTPUT | INPUT_PULLUP

LED_BUILTIN

true | false

Floating Point Constants, Integer Constants

Conversion:

(unsigned int), (unsigned long)

byte(), char(), float(), int(), long(), word()

2.4.1 Data Types:

array, bool, boolean, byte, char, double, float, int, long, short, size_t, string, String(), unsigned char, unsigned int, unsigned long, void, word,

Variable Scope and Qualifiers:

const, scope, static, volatile, Utilities, PROGMEM, sizeof().

2.5 Structure:

The elements of Arduino (C++) code: Sketch, loop(), setup()

Control Structure: break, continue, do...while, else, for, goto, if, return, switch...case, while,

Further Syntax:

#define (define), #include (include) /* */ (block comment), // (single line comment)
; (semicolon), {} (curly braces)

Arithmetic Operators:

% (remainder), * (multiplication), + (addition), - (subtraction), / (division)
= (assignment operator)

Comparison Operators:

== (equal to), != (not equal to), < (less than), <= (less than or equal to)
> (greater than), >= (greater than or equal to)

Boolean Operators:

! (logical not), && (logical and), || (logical or)

Pointer Access Operators:

& (reference operator), * (dereference operator)

Bitwise Operators:

& (bitwise and), << (bitshift left), >> (bitshift right),
^ (bitwise xor), | (bitwise or), ~ (bitwise not)

Compound Operators:

%= (compound remainder), &= (compound bitwise and)
= (compound multiplication), ++ (increment), += (compound addition)
- (decrement), -= (compound subtraction), /= (compound division)
^= (compound bitwise xor), |= (compound bitwise or)

Chapter 3

Example Experiments with Arduino

Arduino IDE comes with a number of example programs which are available from the File/Examples menu on the IDE. We'll use some of these readymade programs to familiarize ourselves with the use of Arduino boards.

3.1 Bare Minimum code example

The bare minimum of code needed to start an Arduino sketch:

This example contains the bare minimum of code you need for a sketch to compile properly on Arduino Software (IDE): the `setup( )` method and the `loop( )` method.

3.1.1 Hardware Required

Arduino Board

3.1.2 Circuit

Only your Arduino Board is needed for this example.

3.1.3 Code

The bare minimum of code needed to start an Arduino sketch:

This example does nothing – it just illustrates the software structure of a sketch developed for Arduino. It contains the bare minimum of code you need for a sketch to compile properly on Arduino Software (IDE): the `setup( )` method and the `loop( )` method.

This code won't actually do anything, but it's structure is useful for copying and pasting to get you started on any sketch of your own. It also shows you how to include comments in your code.

```
void setup ( ) {  
  // put your set up code here to run once:  
}  
  
void loop ( ) {  
  // put your main code here, to run repeatedly:  
}
```

The `setup()` function is called when a sketch starts. Use it to initialize variables, pin modes, start using libraries, etc. The setup function will only run once, after each powerup or reset of the board.

After creating a `setup()` function, the `loop()` function does precisely what its name suggests, and is called repetitively from an endless loop. The function allows your program to change and respond as it runs. Code in the `loop()` section of your sketch is used to actively control the board.

Any line that starts with two slashes (`//`) will not be read by the compiler, so you can write anything you want after it. The two slashes may be put after functional code to keep comments on the same line. Commenting your code like this can be particularly helpful in explaining, both to yourself and others, how your program functions step by step.

3.2 Blinking an LED:

This example shows the simplest thing you can do with an Arduino to see physical output: it blinks the on-board LED.

3.2.1 Hardware Required

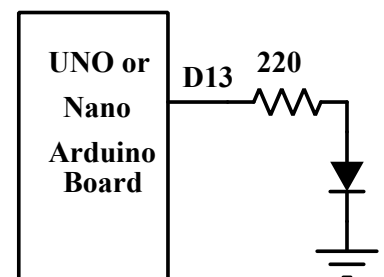
Arduino Board, and (optional) LED, 220 Ω resistor

3.2.2 Circuit

This example uses the built-in LED that most Arduino boards have. The LED is connected to a digital pin and the pin number may vary from board type to board type. To make your life easier, we have a constant that is specified in every board descriptor file. This constant is `LED_BUILTIN` and allows you to control the built-in LED easily. Here is the correspondence between the constant and the digital pin:

Boards 101 Due, Intel Edison, Intel Galileo Gen2, Leonardo and Micro, LilyPad, LilyPad USB, MEGA2560, Mini, Nano, Pro, Pro Mini, UNO, Yún and Zero connect D13 to the LED. Gemma board connects the LED to D1 and MKR1000 connects it to D6.

If you want to light an external LED with this sketch, you need to build this circuit, where you connect one end of the resistor to the digital pin correspondent to the `LED_BUILTIN` constant. Connect the long leg of the LED (the positive leg, called the anode) to the other end of the resistor. Connect the short leg of the LED (the negative leg, called the cathode) to the GND. In the diagram below we show an UNO board that has D13 as the `LED_BUILTIN` value.



The value of the resistor in series with the LED may be of a different value than 220 Ω ; the LED will light up also with values up to 1K Ω .

3.2.3 Code

```
/*
```

```
  Blink Turns an LED on for one second, then off for one second, repeatedly.
```

```
  Most Arduinos have an on-board LED you can control. On the UNO, MEGA and ZERO
```

```

it is attached to digital pin 13, on MKR1000 on pin 6. LED_BUILTIN is set to
the correct LED pin independent of which board is used.
If you want to know what pin the on-board LED is connected to on your Arduino
model, check the Technical Specs of your board at:
https://www.arduino.cc/en/Main/Products
*/

// the setup function runs once when you press reset or power the board
void setup( ) {
  // initialize digital pin LED_BUILTIN as an output.
  pinMode(LED_BUILTIN, OUTPUT);
}

// the loop function runs over and over again forever
void loop( ) {
  digitalWrite(LED_BUILTIN, HIGH); // turn the LED on (HIGH is the voltage level)
  delay(1000);                      // wait for a second
  digitalWrite(LED_BUILTIN, LOW);  // turn the LED off by making the voltage LOW
  delay(1000);                      // wait for a second
}

```

3.3 Digital Read Serial

Read a switch, print the state out to the Arduino Serial Monitor.

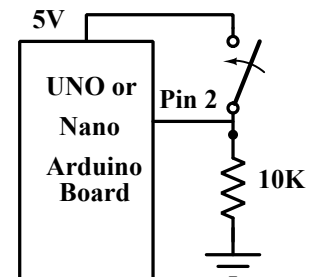
This example shows you how to monitor the state of a switch by establishing serial communication between your Arduino and your computer over USB.

3.3.1 Hardware Required

Arduino Board, a momentary switch, button, or toggle switch, 10K Ω resistor, hook-up wires and breadboard.

3.3.2 Circuit

When the push button is open (unpressed) there is no connection between the two legs of the push button, so the pin is connected to ground (through the 10K Ω pull-down resistor) and reads as LOW, or '0'. When the button is closed (pressed), it makes a connection between its two legs, connecting the pin to 5 volts, so that the pin reads as HIGH, or '1'.



3.3.3 code

In the program below, the very first thing that you do (in the setup function) is to begin serial communications, at 9600 bits of data per second, between your board and your computer with the line:


```
Serial.begin(9600);
```

Next, initialize digital pin 2, the pin that will read the output from your button, as an input:

```
pinMode(2,INPUT);
```

Now that your setup has been completed, move into the main loop of your code. When your button is pressed, 5 volts will freely flow to pin 2 and when it is not pressed, pin 2 will be connected to ground through the 10 K Ω resistor. This is a digital input, meaning that the switch can only be in either an on state (seen by your Arduino as a '1', or HIGH) or an off state (seen by your Arduino as a '0', or LOW), with nothing in between.

The first thing you need to do in the main loop of your program is to establish a variable to hold the information coming in from your switch. Since the information coming in from the switch will be either a '1' or a '0', you can use an int datatype. Call this variable 'sensorValue', and set it to equal whatever is being read on digital pin 2. You can accomplish this with just one line of code:

```
int sensorValue = digitalRead(2);
```

Once the board has read the input, make it print this information back to the computer as a decimal value. You can do this with the command `Serial.println( )` in our last line of code:

```
Serial.println(sensorValue);
```

Now, when you open your Serial Monitor in the Arduino Software (IDE), you will see a stream of '0's if your switch is open, or '1's if your switch is closed.

```
/*
  DigitalReadSerial
  Reads a digital input on pin 2, prints the result to the Serial Monitor
*/

// digital pin 2 has a push button attached to it. Give it a name:
int pushButton = 2;

// the setup routine runs once when you press reset:
void setup( ) {
  // initialize serial communication at 9600 bits per second:
  Serial.begin(9600);
  // make the pushbutton's pin an input:
  pinMode(pushButton, INPUT);
}

// the loop routine runs over and over again forever:
void loop( ) {
  // read the input pin:
  int buttonState = digitalRead(pushButton);
  // print out the state of the button:
  Serial.println(buttonState);
  delay(1);          // delay in between reads for stability
}
```

3.4 Fading a LED

This example demonstrates the use of analog output to fade an LED. We use of the `analogWrite( )` function to fade-in and fade-out an LED.

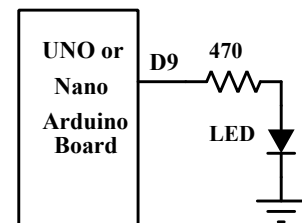
`AnalogWrite` uses pulse width modulation (PWM). In PWM, we turn something on and off very quickly, controlling the ratio of on and off durations. The ratio of the two durations determines the average value at the output. Thus we can generate an average value on the output pin which is not the supply voltage or ground, but can be adjusted to any value in between. Hence it is called “`analogWrite`”. We can use this technique to create a fading effect.

3.4.1 Hardware Required

Arduino board, LED, 220 Ω resistor, hook-up wires, breadboard,

3.4.2 Circuit

Connect the anode (the longer, positive leg) of your LED to digital output pin 9 on your board through a 220 Ω resistor. Connect the cathode (the shorter, negative leg) directly to ground.



3.4.3 Code

After declaring pin 9 to be your LED Pin, there is nothing to do in the `setup( )` function of your code. The `analogWrite( )` function that you will be using in the main loop of your code requires two arguments: One telling the function which pin to write to, and one indicating what PWM value to write.

In order to fade your LED off and on, gradually increase your PWM value from 0 (all the way off) to 255 (all the way on), and then back to 0 once again to complete the cycle. In the sketch below, the PWM value is set using a variable called ‘brightness’. Each time through the loop, it increases by the value of the variable ‘fadeAmount’. If brightness is at either extreme of its value (either 0 or 255), then fadeAmount is changed to its negative. In other words, if fadeAmount is 5, then it is set to -5. If it’s -5, then it’s set to 5. The next time through the loop, this change causes brightness to change direction as well.

The function `analogWrite( )` can change the PWM value very fast, so the delay at the end of the sketch controls the speed of the fade. Try changing the value of the delay and see how it changes the fading effect.

```
/*
  Fade

  This example shows how to fade an LED on pin 9 using the analogWrite( )
  function.

  The analogWrite( ) function uses PWM, so if you want to change the pin
  you're using, be sure to use another PWM capable pin. On most Arduinos,
  the PWM pins are identified with a "~" sign, like ~3, ~5, ~6, ~9, ~10
  and ~11.

  */
```

```

int led = 9;           // the PWM pin the LED is attached to
int brightness = 0;    // how bright the LED is
int fadeAmount = 5;    // how many points to fade the LED by

// the setup routine runs once when you press reset:
void setup( ) {
  // declare pin 9 to be an output:
  pinMode(led, OUTPUT);
}

// the loop routine runs over and over again forever:
void loop( ) {
  // set the brightness of pin 9:
  analogWrite(led, brightness);

  // change the brightness for next time through the loop:
  brightness = brightness + fadeAmount;

  // reverse the direction of the fading at the ends of the fade:
  if (brightness <= 0 || brightness >= 255) {
    fadeAmount = -fadeAmount;
  }
  // wait for 30 milliseconds to see the dimming effect
  delay(30);
}

```

3.5 Reading an analog voltage

This example shows you how to read an analog input on analog pin 0, convert it into voltage, and print it out to the serial monitor of the Arduino Software (IDE).

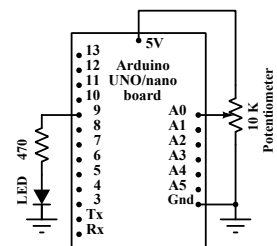
3.5.1 Hardware Required

Arduino Board, 10K Ω potentiometer.

3.5.2 Circuit

Connect three wires from the potentiometer to your board. The first goes to ground from one of the outer pins of the potentiometer. The second goes to 5 volts from the other outer pin of the potentiometer. The third goes from the middle pin of the potentiometer to analog input 0. By turning the shaft of the potentiometer, you change the amount of resistance on either side of the wiper which is connected to the center pin of the potentiometer. This changes the voltage at the center pin, which will be read using an analog to digital converter.

When the resistance between the center and the side connected to 5 volts is close to zero (and the resistance on the other side is close to 10 K Ω), the voltage at the center pin nears 5 volts. When the resistances are reversed, the voltage at the center pin nears 0 volts, or ground. This voltage is the analog voltage that you're reading as an input.



The microcontroller of the board has a circuit inside called an analog-to-digital converter or ADC that reads this changing voltage and converts it to a number between 0 and 1023. When the shaft is turned all the way in one direction, there are 0 volts going to the pin, and the input value is 0. When the shaft is turned all the way in the opposite direction, there are 5 volts going to the pin and the input value is 1023. In between, `analogRead( )` returns a number between 0 and 1023 that is proportional to the amount of voltage being applied to the pin.

3.5.3 Code

In the program below, the very first thing that you do will in the setup function is to begin serial communications, at 9600 bits of data per second, between your board and your computer with the line:

```
Serial.begin(9600);
```

Next, in the main loop of your code, you need to establish a variable to store the resistance value (which will be between 0 and 1023, perfect for an int datatype) coming in from your potentiometer:

```
int sensorValue = analogRead(A0);
```

To change the values from 0-1023 to a range that corresponds to the voltage the pin is reading, you'll need to create another variable, a float, and do a little math. To scale the numbers between 0.0 and 5.0, divide 5.0 by 1023.0 and multiply that by `sensorValue` :

```
float voltage= sensorValue * (5.0 / 1023.0);
```

Finally, you need to print this information to your serial window. You can do this with the command `Serial.println( )` in your last line of code:

```
Serial.println(voltage)
```

Now, when you open your Serial Monitor in the Arduino IDE (by clicking on the icon on the right side of the top green bar or pressing Ctrl+Shift+M), you should see a steady stream of numbers ranging from 0.0 - 5.0. As you turn the pot, the values will change, corresponding to the voltage coming into pin A0.

```
/*
  ReadAnalogVoltage
  Reads an analog input on pin 0, converts it to voltage,
  and prints the result to the Serial Monitor.
  Graphical representation is available using Serial Plotter
  (Tools > Serial Plotter menu).
  Attach the center pin of a potentiometer to pin A0, and
  the outside pins to +5 V and ground.
*/
// the setup routine runs once when you press reset:
void setup( ) {
  // initialize serial communication at 9600 bits per second:
  Serial.begin(9600);
}

// the loop routine runs over and over again forever:
void loop( ) {
  // read the input on analog pin 0:
  int sensorValue = analogRead(A0);
  //Convert the analog reading (which goes from 0 - 1023) to a voltage (0 - 5 V):
  float voltage = sensorValue * (5.0 / 1023.0);
  // print out the value you read:
```

```

    Serial.println(voltage);
}

```

3.6 Reading the timer

Sometimes you need to do two things at once. For example you might want to blink an LED while reading a button press. In this case, you can't use `delay( )`, because Arduino pauses your program during the `delay( )`. If the button is pressed while Arduino is paused waiting for the `delay( )` to pass, your program will miss the button press.

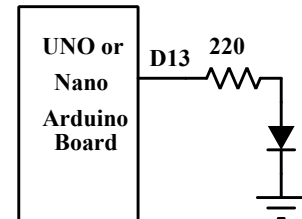
This sketch demonstrates how to blink an LED without using `delay( )`. It turns the LED on and then makes note of the time. Then, each time through `loop( )`, it checks to see if the desired blink time has passed. If it has, it toggles the LED on or off and makes note of the new time. In this way the LED blinks continuously while the sketch execution never lags on a single instruction.

3.6.1 Hardware Required

Arduino Board, LED, 220 Ω resistor.

3.6.2 Circuit

To build the circuit, connect one end of the resistor to pin 13 of the board. Connect the long leg of the LED (the positive leg, called the anode) to the other end of the resistor. Connect the short leg of the LED (the negative leg, called the cathode) to the board GND, as shown in the schematic. Or else, you can just use the inbuilt LED on the board.



3.6.3 Code

The code uses the `millis( )` function, a command that returns the number of milliseconds since the board started running its current sketch, to blink an LED.

```

/*
  Blink without Delay

  Turns on and off a light emitting diode (LED) connected to a digital pin,
  without using the delay( ) function. This means that other code can run
  at the same time without being interrupted by the LED code.

  The circuit:
  - Use the onboard LED.
  - Note: Most Arduinos have an on-board LED you can control. On the UNO, MEGA
    and ZERO it is attached to digital pin 13, on MKR1000 on pin 6.
    LED_BUILTIN is set to the correct LED pin independent of which board is
    used. If you want to know what pin the on-board LED is connected to on
    your Arduino model, check the Technical Specs of your board at:
    https://www.arduino.cc/en/Main/Products
*/

```

```

// constants won't change. Used here to set a pin number:
const int ledPin = LED_BUILTIN; // the number of the LED pin

// Variables will change:
int ledState = LOW;             // ledState used to set the LED

// Generally, you should use "unsigned long" for variables that hold time
// The value will quickly become too large for an int to store
unsigned long previousMillis = 0; // will store last time LED was updated

// constants won't change:
const long interval = 1000;     // interval at which to blink (milliseconds)

void setup( ) {
  // set the digital pin as output:
  pinMode(ledPin, OUTPUT);
}

void loop( ) {
  // here is where you'd put code that needs to be running all the time.

  // check to see if it's time to blink the LED; that is, if the difference
  // between the current time and last time you blinked the LED is bigger than
  // the interval at which you want to blink the LED.
  unsigned long currentMillis = millis( );

  if (currentMillis - previousMillis >= interval) {
    // save the last time you blinked the LED
    previousMillis = currentMillis;

    // if the LED is off turn it on and vice-versa:
    if (ledState == LOW) {
      ledState = HIGH;
    } else {
      ledState = LOW;
    }

    // set the LED with the ledState of the variable:
    digitalWrite(ledPin, ledState);
  }
}

```

MS101 Makerspace

Expt. 4: Arduino Familiarization and Motor Control

Department of Electrical Engineering
Indian Institute of Technology, Bombay

February 2025

Arduino Nano Board

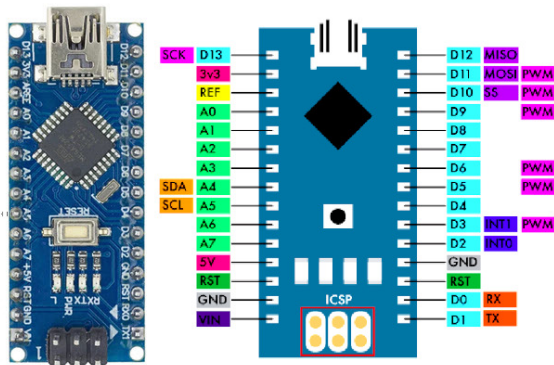


Diagram showing pins

Some of the inexpensive nano boards need special drivers (CH 340) with Windows. The driver can be downloaded from:

https://www.wch-ic.com/downloads/CH341SER_ZIP.html

A tutorial for installing it is available at:

<https://learn.sparkfun.com/tutorials/how-to-install-ch340-drivers/all>

Arduino Nano uses a USB mini connector to connect to a laptop/PC.

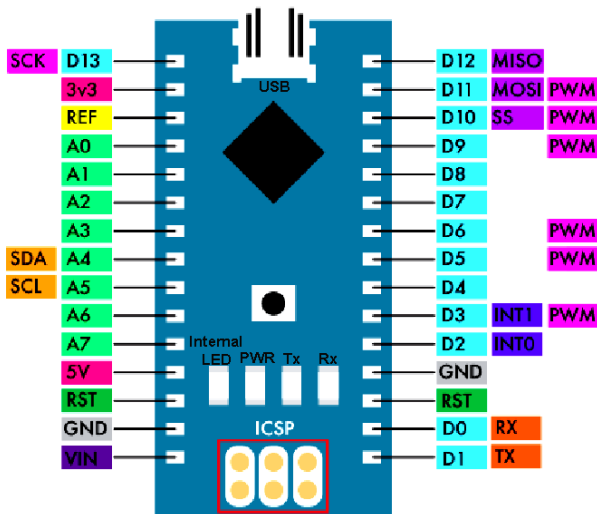
It derives power through the USB.

The board can be plugged directly into a breadboard for trial circuits.

Powering up the Arduino board

Power to the Arduino Nano board may be supplied:

- 1 Through the USB connector.
(Nano uses a mini USB connector)
- 2 Through the Vin pin (bottom left in the figure to the right).



The “power on” LED will light up when power is applied.

Programming Arduino Boards

- Arduino boards are programmed using an Integrated Development Environment (IDE) running on a laptop or PC.
- Detailed instructions were put up on moodle for installing the IDE depending on your operating system.
- **YOU SHOULD HAVE INSTALLED THE IDE ON YOUR LAPTOP BEFORE COMING TO THE LAB.**

Note that the current version of the IDE is version 2.3.0.

- In order to communicate with your board, the IDE needs to know
 - 1 The type of your board (Uno/Nano/...), and
 - 2 The serial line you will use for connecting the card to the laptop/PC.
- The write up on moodle provides all details which are required for setting up these two parameters for the IDE.

Programming Arduino Boards: Some terminology

Arduino programs which follow a pattern suitable for event-driven applications are called *sketches*.

- A sketch has two parts – the initialization part which is run first (and only once), and the repetitive part which runs in a perpetual loop.
- You need to define two functions (called “setup” and “loop”) which provide these two parts. These functions receive no parameters from the calling code and return nothing to the caller (void type).
- These functions are to be written in Arduino flavour of C or C++.
- Many pre-defined functions for specific actions are available as libraries. These functions can be called from your **setup()** or **loop()** code, which makes it easy to write software for applications.

Experiments for Lab. 4

This experiment has two parts: part A is for familiarization with Arduino board, its Integrated Development Environment (IDE) and software. Part B is for controlling motors using Arduino boards.

In part 4A, we shall carry out 5 simple experiments using Arduino.

- 1 LED Blink experiment for Digital output – to be done before coming to the lab to test your board and IDE installation.
- 2 LED fade experiment for Pulse Width Modulated outputs.
- 3 Switch reading (digital input) and serial output,
- 4 Reading voltage output from a potentiometer as an illustration of analog input.

For this lab, we'll use ready made example programs available in the Arduino IDE with minimal modifications.

Experiments for Lab. 4

Before starting the experiments, it is assumed that

- 1 you have read this document and an earlier file (Introduction to Arduino) that we had put on moodle.
- 2 You have acquired an Arduino board.
- 3 You have installed and checked the IDE according to your operating system.
- 4 You have checked the Arduino board by connecting it to your laptop using an appropriate type of cable – USB A on the laptop side and USB B on micro-controller side for Arduino Uno and a mini USB connector for Arduino Nano.
- 5 You have run the blink experiment (described next) **before** coming to the lab.

Connect the board to your laptop through the USB cable and start the IDE according to the conventions of your OS (for example, by double clicking on the program name).

Expt. 4 – IDE set up

- In the IDE, click on the down arrow in the window labelled as “Select Board” and choose to “Select Other Board and Port”.
- In the search window, type Nano and the board option for Arduino Nano will come up. Click on it and a tick mark will appear against the name.
(You could also scroll down the alphabetical list of boards to select your board).
- Click on the serial port shown on the right half of the window. A tick mark will appear here as well. Click on OK to select these choices.
- The code for a previously run program may be already displayed in the editing window. However, we shall begin with a fresh ready-made program.

Expt. 4A-1 – Digital Output and delay: Blinking an LED

Objective: Familiarization with Arduino and its IDE.

This experiment does not require any external circuit or instruments. You should perform it **before** coming to the lab to check that your board is working properly and the IDE is installed correctly.

- Click on the File tab and choose “Examples->01-Basics->Blink. A fresh edit window will open with the sketch for Blink loaded in it.
- Read through the comments to understand what the program is supposed to do and notice the function calls used.
- Click on the “Sketch” tab and choose to verify/compile. The IDE responds with “Done compiling”. (A short cut for this is control-R).
- Click on the “Sketch” tab again and choose to “Upload”. The IDE responds with “Done uploading”. (A short cut for this is control-U).
- The → icon on top can be used to compile and upload in one go.
- The Blink program starts running. Admire your handiwork lovingly.

Expt. 4A – Digital Output and delay: Blinking an LED

Now we'll tinker with the sketch . . .

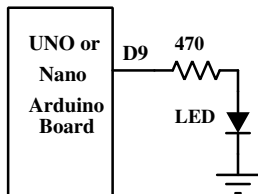
This part will be done in the lab.

- Change the on-time/off-time to 500 ms/1500 ms in the sketch by modifying the appropriate numbers in the code.
- Compile and upload again and run it to observe a different ratio of on and off time for the LED.
- Change the on time/off-time to 1500 ms/500 ms, recompile, upload and run.
- Call your TA to participate in your excitement . . .
(who may spoil the fun somewhat by asking you about the function calls used and the structure of the program).
- Have your lab book signed for completion of Experiment 4A-1.

Expt. 4A-2 – Pulse Width Modulation: Fade

Objective: Use of Pulse width modulation (PWM) to control power given to an external device.

This experiment requires making a small circuit on your breadboard.



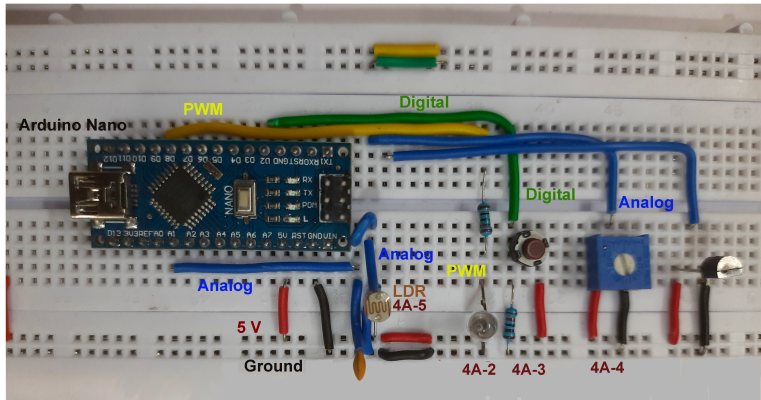
The on-board LED is connected to pin 13, which does not support Pulse Width Modulation. Hence the need for this circuit.

- Connect a wire from Digital pin D-9 on your UNO/Nano board to the breadboard.
- Connect a resistor (any value between 220Ω to 470Ω will do) from this point to the anode of an LED (its longer lead).
- Return the cathode of the LED (its shorter lead) to a long line of the breadboard for the ground connection.
- Connect a (black) wire from this long line to the GND terminal of Arduino (in the group of pins labelled as “power”).

Done!

Expt. 4A-2 – Pulse Width Modulation: Fade

This is how I wired up all parts of expt. 4 on a breadboard, using a Nano board.

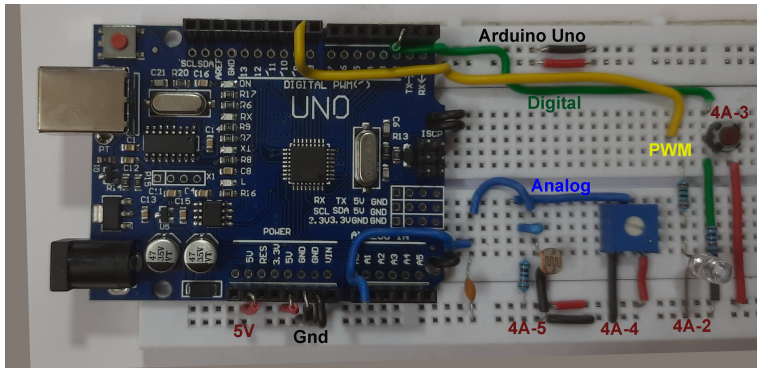


(Beware – due to parallax, wires may appear to go to the wrong pins on Arduino – follow the text, not what you perceive from the figure.)

Wire colours: Red: 5V, Black: Ground,
Green: Digital, Yellow: PWM, Blue: Analog

Expt. 4 with an Uno board

The photograph shows all parts of expt. 4 wired up using an Uno board.



(Beware – due to parallax, wires may appear to go to the wrong pins on Arduino – follow the text, not what you perceive from the figure.)

Wire colours: Red: 5V, Black: Ground,
Green: Digital, Yellow: PWM, Blue: Analog

Expt. 4A-2 – Pulse Width Modulation: Fade

- Click on the File tab and choose “Examples->01-Basics->Fade”. A fresh edit window will open with the sketch for Fade loaded in it.
- Read through the comments to understand what the program is supposed to do and notice the function calls used.
- Click on the “Sketch” tab and choose to verify/compile. The IDE responds with “Done compiling”.
- Click on the “Sketch” tab again and choose to “Upload”. The IDE responds with “Done uploading”.
- Again, you could click on the → icon to combine the compile and upload operations.
- The Fade program starts running.
Watch the LED on the breadboard fade in and out. View the waveform on pin D9 using the Digital Storage Oscilloscope (DSO).

Expt. 4A-2 – Pulse Width Modulation: Fade

- Modify the sketch to use a different fading increment.
- Re-compile, upload and run. Watch the effect of slower and faster fading on the LED.
- Define separate values for increment/decrement amount so that the rate of fading is not symmetrical during fade in and fade out phases. (For this, you will have to modify the program code).

For code modification, you will define two values for fade amounts in the `setup( )` function.

```
int brightness = 0; // how bright the LED is
int fd1 = 5; // how many points to fade the LED by
int fd2 = 20;
int fadeAmount = fd1; // Initial fading rate
```

(Only the changed part of the program has been described above)

Expt. 4A-2 – Pulse Width Modulation: Fade

In the `loop( )` function, you have to check for two termination conditions:

```
//change the brightness for next time:
brightness = brightness + fadeAmount;
// reverse the direction of the fading
// when you reach max brightness
if (brightness > 255) {
    brightness = 255-fd2;
    fadeAmount = -fd2;
}

if (brightness < 0) {
    brightness = fd1;
    fadeAmount = fd1;
}
// wait for 50 milliseconds per brightness change
delay(50);\\
```

Expt. 4A-2 – Pulse Width Modulation: Fade

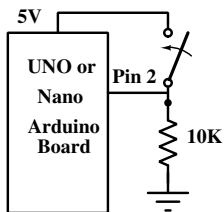
After modifying the code,

- Re-compile, re-upload, re-run, and rejoice.
- View the waveform on pin 9 using the DSO.
- Draw this in your note book. (Use waveforms with dotted/dashed lines to show how the pulse width is changing).
- Call the RA/TA to show your program running with asymmetrical fading rate and get the completion of Experiment 4A-2 signed.

Do not dismantle this circuit when you go to the next part!

Expt. 4A-3 – Digital Input, Serial Output

We need to make another simple circuit for this experiment.



Digital pin 2 will be used as the digital input. The circuit shows a switch, but we can simulate the switch by just touching a wire manually.

Connect the ground pin (among the power pins of Arduino) to the long ground line of your breadboard. Plug in a $10\text{K}\Omega$ resistor between ground and any group of shorted 5 pins.

(Any value between $10\text{K}\Omega$ and $100\text{K}\Omega$ will do)

Connect a wire from digital pin 2 to the top of this resistor. Connect a wire from the 5V output in the group of pins marked “power” on Arduino.

We'll touch the other end of this wire by hand to the top of the resistor to simulate a switch.

Expt. 4A-3 – Digital Input, Serial Output

- Click on the File tab and choose “Examples->01-Basics->DigitalReadSerial”. A fresh edit window will open with the sketch for DigitalReadSerial loaded in it.
- Read through the comments to understand what the program is supposed to do and notice the function calls used. Pay particular attention to the way the serial link will be established between the laptop and Arduino.
- Click on the “Sketch” tab and choose to verify/compile. The IDE responds with “Done compiling”.
- Click on the “Sketch” tab again and choose to “Upload”. The IDE responds with “Done uploading”.

Expt. 4A-3 – Digital Input, Serial Output

- To see the measured values, open the serial monitor.
For this, click on the Tools tab and then on the Serial Monitor choice. This will open a window below the edit window to show serial communication between the Laptop and Arduino.
- Watch the string of '0's appearing in the serial monitor. This is because Arduino is reading a Low voltage on pin 2 through the grounded 10K Ω resistor.
- Press the push button (or touch the wire you had connected from the 5V output pin on Arduino to the top of 10K Ω resistor). You will see a series of '1's now in the serial monitor window.

On the top right of IDE window are two icons.

These can be used to invoke the digital monitor and digital plotter provided by the IDE.

Expt. 4A-3 – Digital Input, Serial Output

- Press and release the push button (or touch and remove the 5V wire from the top of the 10K Ω resistor) and check that the digital value read on pin 2 changes from '0' to '1' and back.
- Call your TA and claim that you have learnt:
 - 1 how to read digital input,
 - 2 how to send data through the serial line, and
 - 3 how to plot incoming data from Arduino on the laptop.

(The TA may be tempted to test whether you have actually learnt it)!

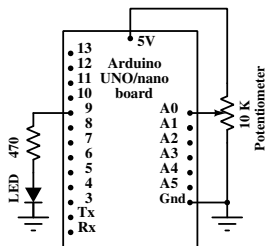
Get your lab book signed for completion of Experiment 4A-3.

Expt. 4A-4 – Analog Input, Serial Output

Another experiment . . . another circuit.

You are now veterans of making circuits on the bread board.

So just make the circuit shown below without detailed instructions.



- Do not forget to connect the 5V and Ground lines to the 5V and ground pins in the “power” group of pins on the Arduino board.
- The breadboard compatible potentiometer covers the nearby pins – so make a note of the label on the group of 5 pins on the breadboard to which each of its terminals is connected.

The LED connected to pin D9 through a (≈ 470) Ω resistor should still be there. (If you removed these by mistake, connect these again as shown).

Expt. 4A-4 – Analog Input, Serial Output

- Click on the File tab and choose “Examples->03.Analog->AnalogInOutSerial.
A fresh edit window will open with the sketch for AnalogInOutSerial loaded in it.
- Read through the comments to understand what the program is supposed to do and notice the function calls used.
- Pay particular attention to the way the range of 10bit ADC on AVR 328P is mapped to a range of 8bits (0 to 255), and how a serial link is established between the laptop and Arduino.
- Click on the “Sketch” tab and choose to verify/compile. The IDE responds with “Done compiling”. (Short cut: control-R).
- Click on the “Sketch” tab again and choose to “Upload”. (Short cut: control-U). The IDE responds with “Done uploading”.

Expt. 4A-4 – Analog Input, Serial Output

- Open the serial monitor. For this, click on the Tools tab and then on the Serial Monitor choice. (You can also click on the icon at the top right of the IDE). This will open a window below the edit window to show serial communication between the Laptop and Arduino.
- Adjust the position of the potentiometer by rotating the slider using a small screw driver. Watch the intensity of the LED being controlled by the position of the potentiometer.
- Also see the sensor output values (ADC outputs with a 10 bit range and the mapped values to 8 bit ranges) being sent through the serial port. These appear in the serial monitor. You can also use the plotter function to see how these change when you turn the potentiometer slider.

Show the operation to your TA and explain how it all works. Get your notebook signed for observations of experiment 4A-4.

Motor Speed Control with PWM using an Arduino Board

In this part of the experiment, we'll learn how to control the speed of a motor using an Arduino board.

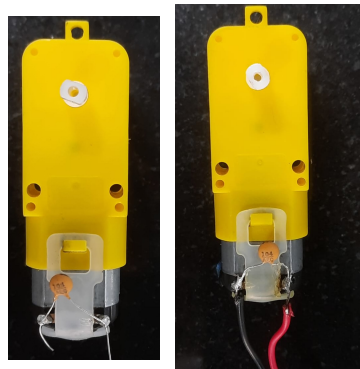
We need a motor driver card and battery operated (BO) motors for this part.

First, we have to assemble the set up. The various steps for this are:

- Soldering a capacitor and wires to the Battery Operated (BO) motor.
- Mounting the motor on a stage and fixing an encoder disk on its axis.
- Mounting a reflective sensor with an LED and photo-diode which gives digital outputs of '1'/'0' as the encoder disk rotates.
- Connecting the motor to its driver and connecting the driver to Arduino Nano card.

Soldering a Capacitor across BO Motor Terminals

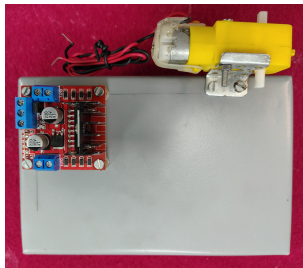
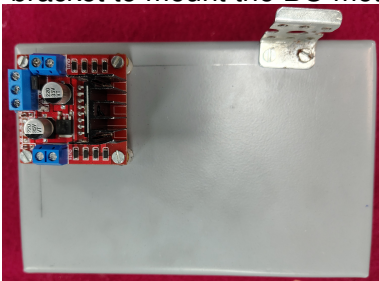
- Before we connect the motor to our circuits, we need to solder a $0.1\mu\text{F}$ ceramic capacitor across its terminals to suppress noise.
- We also need to solder wires to the motor terminals, so that we can connect it to the motor driver.



Solder the capacitor and wires to a BO motor.
Seek help from your TA/EE staff if you are unable to solder these neatly.

Mounting the motor on a platform

You are given a platform with the motor driver mounted on it and a bracket to mount the BO motor.

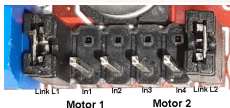
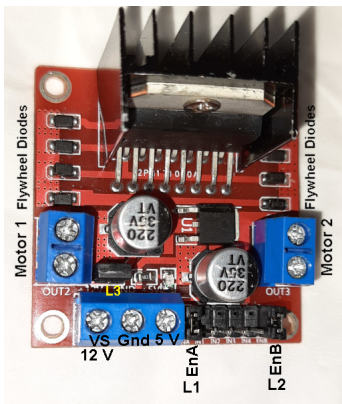


Platform with mounted motor driver BO motor mounted on the platform and bracket

- Mount the BO motor on the bracket using nuts and bolts provided to you, as shown in the figure above.
- If your motor has a single shaft, it should be pointing outwards.
- Wires soldered to the motor will be connected to the motor driver board.

Configuring the Motor Driver Board

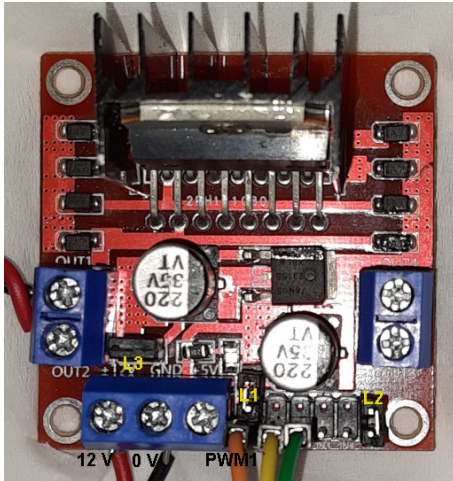
We shall use the motor driver L298. This board can drive two BO motors, but we are going to use only one motor in this experiment.



The board has 3 “**links**”. (A link is a little plastic part with metal inside, used to connect two pins to each other). Locate the three links:

- 1 Link L1 is located just to the right of the power block on the bottom, between the power block and In1 connection.
- 2 Link L2 is at the bottom right just above the PWM2 pin.
- 3 Link L3 is at left, just above the power block and to the right of OUT2 motor connection.

Using Links to configure the motor driver



While removing a link, if you just take it off, you are likely to lose it.

To remove a link, pull it out and plug it back such that it covers only one of the two pins over which it was originally placed.

(See link L1 which has been moved up by one position.

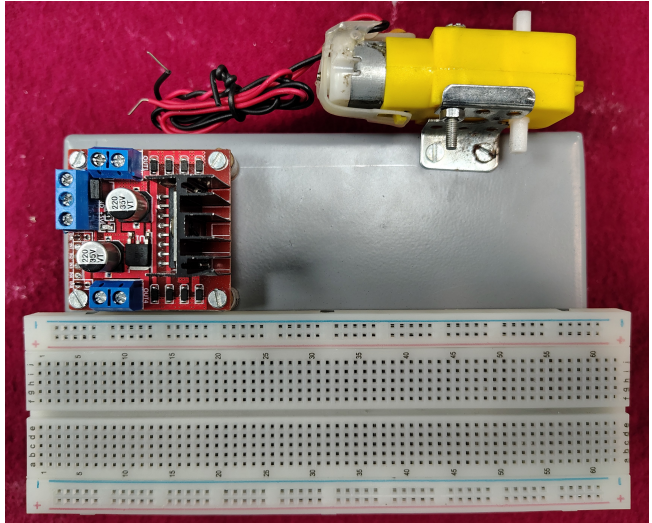
This exposes the pin previously shorted by the link and now has an orange wire labelled PWM1 connected to it.)

Breadboard for connecting Arduino Nano

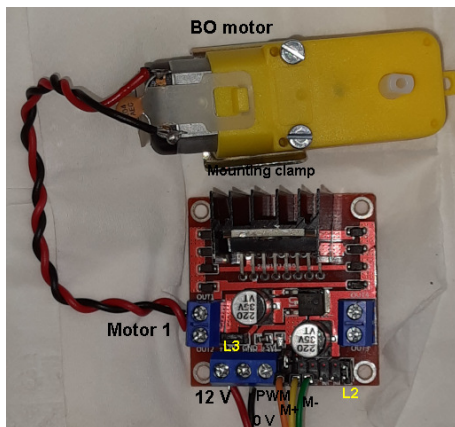
We shall use an Arduino Nano board for controlling the motor.

The Nano card will be plugged into a breadboard, which will be placed on the platform as shown in the figure.

The breadboard can be stuck to the platform using a double sided sticking tape.

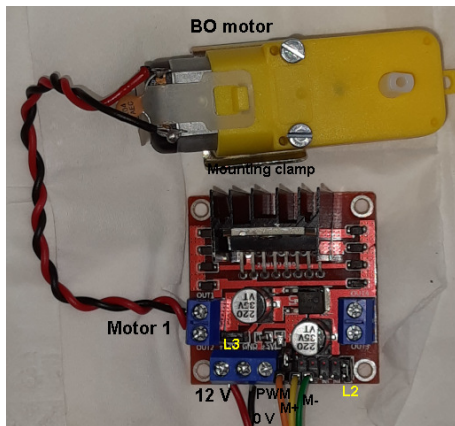


Connecting Motor and Power to the Driver Board



- The motor leads (soldered earlier) should be connected to Motor 1 terminals (OUT1 and OUT2) on the L298 motor driver board.
- Connect a DC power supply (programmed to 12 V) to the V_S and Ground terminal blocks on the L298 card. The third terminal (meant for 5 V DC) **should be left open**.
- Connect the ground terminal of L298 card to the ground pin of Arduino.

Connections between Motor Driver and Arduino



- Remove Link L1. (The link connects the enable input for motor A to 5 V generated by on-board 5 V regulator).
- The pin (which was connected to on-board 5 V through the link earlier) should now be connected to pin 6 of Arduino for PWM control.
- Connect the next two pins (IN1 and IN2) on the motor driver to pins 10 and 11 of Arduino. (Yellow and Green wires in the picture).

Female to male hook up wires should be used for these connections.

Connection Check List

Links: L3 in; L1 removed.

Power:

Driver card	Power supply
V_S (10-12 V)	Supply+
Ground	Supply–
5 V Terminal	Open

Motor to Driver card

OUT1, OUT2 - Motor wires

Motor card to Arduino

Driver Card	Arduino UNO/nano
Ground	Gnd
Enable A	Pin 6
IN1	Pin 10
IN2	Pin 11

After making all the connections, have these checked by your TA.

Sketch for Rotating the Motor

We are all set to rotate the motor.

We shall use the provided sketch (sketch-expt4B-1.ino) to do this. The sketch defines a PWM value in the code (at the end of the sketch) and runs the motor with this PWM value.

If the defined PWM value is negative, it rotates the motor in the opposite direction.

- Connect your Arduino to the laptop **without powering on the motor driver.**
(Keithley Power supply output should be OFF).
- Enter, compile and upload the provided sketch (sketch-expt4B.ino).

Rotating the Motor

We'll use 12 V power from the Keithley power supply (Channel 1 or 2).

- Set the voltage on the selected channel to 12 V.
- Program Iset to 1A on this channel and turn ON the output.

The power LED on motor driver should glow.

If it does not, turn everything off and re-check your connections.

If all is well, **your motor will rotate!**

Effect of PWM Value on Motor Speed

When you have stopped jumping for joy, call the TA and show your success to him/her. Have your lab book signed that your motor connections are right and the motor is rotating.

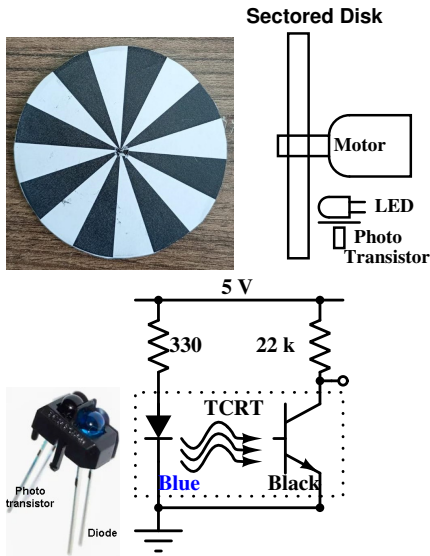
Try various positive and negative values of PWM.

(Absolute value of PWM has to be ≤ 255).

For this you will have to edit the sketch, altering the value of `pwmValue` near the end of the sketch every time, and re-uploading the binary to Arduino.

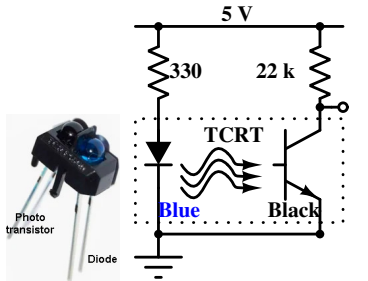
After that, Disconnect the power from the motor driver –
We have more work to do ...

Measuring Rotation Speed

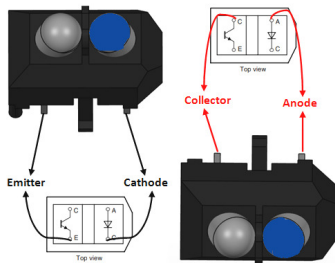


- Mount the encoder disk with black and white sectors on your motor
- We shall use an infrared LED/photo-transistor pair (TCRT sensor) to detect the rotation of this disk.
- The circuit for the TCRT sensor is shown on the left. This circuit is given to you as a pre-soldered PC board.
- You only have to connect the power and ground pins from the PC board to your circuit.

Measuring Rotation Speed



- Connect the power and ground terminals of the TCRT sensor circuit board to 5V and Gnd terminals of Arduino Nano.
- Run the sketch sketch-expt4B-1.ino and Power the motor on as before so that the encoder disk rotates.
- Place the TCRT sensor next to the slots on the rotating encoder disk.
- Observe the waveform at the TCRT output on the oscilloscope.
- Draw the waveform in your record book.



Measuring Rotation Speed

- Due to light beam reflection by rotating sectors on the encoder disk, you will see a (nearly) square waveform on the oscilloscope.
- The time period of this waveform is the time taken for one slot to move into the location of the previous slot.
- Since there are 8 white sectors on the disk, the time for one full rotation is 8 times the time period of the waveform.
- The reciprocal of this is the speed in revolutions per second.
- Speed is often specified in revolutions per minute (RPM). That is just 60 times the revolutions per second that we had computed.

Rotation Speed vs PWM Plot

- Modify the PWM value in the given sketch (sketch-expt4B-1.ino) from 32 to 224 in steps of 32.
- Notice that at low PWM values, the motor may not rotate at all. This is normal.
- Compute Rotation Speed for each value of PWM. Enter these values in your note book as a table and have these signed by your TA.
- Plot Rotation Speed vs the applied PWM.

After this, enter, compile and upload the second sketch given to you. (sketch-expt4B-2.ino).

See the motor accelerate in one direction, decelerate, then accelerate in the other direction and decelerate to idle condition.

See how this is accomplished in the code in the given sketch.

Sample Pre-Lab Quiz Questions for Expt 04

Duration: 10 min; Max marks: 5 marks

Q1 Mark all statements which are correct:

An Arduino Nano/Uno card can be powered

- i) through its USB port from a computer.
- ii) by giving 9V to Aref pin
- iii) by giving a voltage between 7 and 12 V to the Vin pin
- iv) by connecting the output of a 9V wall plug DC source to A0 pin
- v) by connecting a DC source of 7 to 12 V to the Rx pin
- vi) by connecting a DC source of 7 to 12 V to the Tx pin

Q2 Pulse width modulation (PWM) can be used to control the duty cycle of the output. On an Arduino Nano/Uno board, PWM output is available on

- i) all digital output pins D0-13.
- ii) all Analog pins A0 to A5.
- iii) selected Digital output pins.
- iv) serial I-O pins Tx and Rx.

Q3 Mark all statements which are correct.

- i) The USB port is used to supply power to the Nano/Uno board.
- ii) The program binary can be loaded into the flash memory using the USB interface.
- iii) Data output from the microcontroller can be sent to the PC using the USB interface.
- iv) The USB interface is used on the Nano/Uno boards to read programs from a USB drive.
- v) The USB interface is used on the Nano/Uno boards to read data files from a USB drive.

Q4 Mark all statements which are correct.

- i) We put a capacitor across BO motor terminal to cancel the inductance of motor windings.
- ii) A capacitor is soldered across the BO motor terminals to suppress noise.
- iii) A B.O. motor can be driven directly from the digital outputs of Arduino.

- iv) The 5V regulator on the motor driver card is used to drive the motor from a steady voltage.
- v) The 5V regulator on the motor driver board is used to power the digital circuits on the controller.

Q5 Mark all statements which are correct.

- i) The on-board ADC controller has 8 bit resolution.
- ii) The on-board ADC controller has 10 bit resolution.
- iii) Output of the ADC controller can be used directly to control the PWM duty cycle
- iv) Output of the ADC controller should be mapped onto 8 bit range before using it as the PWM value.
- v) AnalogWrite statement outputs an analog voltage on pins A0-A5
- vi) AnalogWrite statement outputs pulse width modulated output on selected digital pins